Design Track

Interview Preparation Series • Professional Edition

API Design

Compiled with LaTeX • June 17, 2026

harsha.somisetty01@gmail.com

In This Edition

> **How to use this file:** API design shows up in two places — as a dedicated "design the API for X" round, and as a 5-minute sub-step inside every system-design interview ("sketch the endpoints"). This guide builds the topic from first principles: the *mechanisms* behind REST semantics, idempotency, pagination, auth, and rate limiting — not just the vocabulary. Read top-to-bottom once for understanding, then drill the `Common Interview Questions` and `Revision Cheat Sheet` before a loop. Every section has a *why*, a concrete example (HTTP/JSON/SQL/proto), and an interview-grade trade-off.

1 Overview --- What It Is

An **API (Application Programming Interface)** is the contract that lets one piece of software talk to another. In the context of this guide, "API design" means the discipline of defining the **surface area** of a network service: which operations exist, how they are addressed, what they accept and return, how errors are reported, how the contract evolves, and how it stays reliable, secure, and performant under load.

A well-designed API is **a product**. Its users are developers — internal teams, mobile clients, third-party partners. Like any product, the quality bar is: *can a competent developer integrate without reading your source code, and will their integration keep working a year from now?*

The contract has several layers:

```
┌──────────────────────────────────────────────────────────┐
│  Semantic layer   "POST /orders creates an order, charges │
│                    the card, returns 201 with a Location."│
├──────────────────────────────────────────────────────────┤
│  Schema layer     request/response shapes, field types,   │
│                    required vs optional, enums           │
├──────────────────────────────────────────────────────────┤
│  Protocol layer   HTTP verbs, status codes, headers /     │
│                    gRPC methods / GraphQL operations     │
├──────────────────────────────────────────────────────────┤
│  Transport layer  HTTP/1.1, HTTP/2, TLS, TCP              │
└──────────────────────────────────────────────────────────┘
```

Properties of a good API:

Property	What it means
Predictable	Consistent naming, verbs, and error shapes — once you learn one endpoint, you can guess the next
Hard to misuse	Required fields enforced, sensible defaults, idempotency for retries
Evolvable	New fields/endpoints can be added without breaking existing clients
Documented	Machine-readable spec (OpenAPI / .proto / GraphQL SDL) plus prose
Observable	Stable error codes, request IDs, rate-limit headers
Secure by default	AuthN/AuthZ on every endpoint, least-privilege scopes, TLS everywhere

First-principles mental model:

```
Use cases → Resources & operations → Contract → Evolution & operability
```

You start from *what clients need to do*, model the *nouns and verbs*, freeze a *contract*, and then spend most of your engineering effort keeping that contract stable while the implementation underneath changes constantly.

2 Why It Exists

In a monolith, modules call each other via in-process function calls: the compiler checks types, calls are instant and reliable, and refactoring is a single atomic commit. The moment two modules live in **different processes, machines, or organizations**, that comfort disappears:

1. **Calls cross a network** → they can be slow, fail, or be delivered twice. The API must define behavior under partial failure (timeouts, retries, idempotency).
2. **Caller and callee deploy independently** → you can no longer change both sides atomically. The API must be a *stable, versioned contract* that tolerates skew.
3. **The caller might be a stranger** → a third-party developer or untrusted mobile client. The API must authenticate, authorize, validate input, and rate-limit.
4. **There is no shared type system** → JSON/Protobuf/SDL schemas replace the compiler as the source of truth.
5. **Many clients, diverse needs** → a TV app, a watch app, and a partner batch job all hit the same backend. The API must serve them without a custom endpoint per client (or it must, deliberately, via BFFs).

Each of these pains birthed a pattern in this guide: idempotency keys (pain #1), versioning (#2), OAuth/JWT/scopes (#3), schemas/OpenAPI (#4), GraphQL and BFFs (#5). Understanding *which pain* a feature solves is what lets you defend it in an interview.

3 Why FAANG Cares

Company	What they're really testing	Specific signals
Amazon	Service-oriented architecture is literally Bezos's 2002 API Mandate ("all teams expose data via service interfaces, no exceptions"). API design *is* the culture.	Idempotency for payments, backward compatibility, fine-grained ownership, designing for partners (AWS APIs are public products)
Google	Has a famous internal *API Design Guide* (AIP — API Improvement Proposals). gRPC and Protobuf are theirs.	Resource-oriented design, standard methods (List/Get/Create/Update/Delete), proto evolution rules, field masks
Meta	Invented GraphQL to solve mobile over/under-fetching across a huge product surface.	GraphQL schema design, N+1 / DataLoader, query cost limiting, versionless evolution
Microsoft	Publishes the *REST API Guidelines*; Azure and Graph API are massive public surfaces.	OData-style query params, consistent error envelope, long-running operations (202 + polling), throttling headers
Netflix	Pioneered the BFF pattern and edge API gateways (Zuul) for hundreds of device types.	Gateway design, per-device aggregation, resilience (timeouts, fallbacks, circuit breakers)
Apple	Privacy-first, App Store APIs, APNs push.	Scoped tokens, minimal data exposure, signed requests, deprecation discipline
Stripe/payments style	The gold standard for developer-facing API design — every interviewer has read their docs.	Idempotency keys, cursor pagination, versioned via date, rich error objects, expandable resources

Universal signal: Can you design a contract that is *correct under retries*, *safe under concurrency*, *evolvable without breaking clients*, and *defensible on every trade-off*? Junior candidates list

verbs; senior candidates reason about idempotency, pagination stability, and deprecation.

4 Core Concepts

4.1 Resource vs RPC vs Query paradigms

There are three dominant API styles. They are not "REST is best" — they solve different problems.

	REST (resource-oriented)	RPC (e.g. gRPC)	Query (GraphQL)
Mental model	Nouns + standard verbs	Functions you call	A typed graph you query
Addressing	URLs (`/orders/42`)	Methods (`OrderService.Get`)	Single endpoint (`/graphql`)
Shape of data	Server decides representation	Server decides	Client picks fields
Best for	Public CRUD-ish APIs, caching	Internal low-latency microservices	Mobile/diverse clients, aggregation
Transport	HTTP/1.1 or 2, JSON	HTTP/2, Protobuf	HTTP, JSON

Interview takeaway: Lead with *who the client is*. Public partner API → REST. Internal service-to-service → gRPC. Many heterogeneous front-ends fetching nested data → GraphQL. Saying "it depends on the consumer" earns more points than dogma.

4.2 Statelessness

Each request must carry **all context needed to process it** (auth token, parameters). The server holds no per-client session in memory between requests. This is a REST constraint and a scaling necessity: any server instance can handle any request, so you can add/remove instances freely behind a load balancer. State lives in tokens (client-side) or shared stores (Redis/DB), never in app-server memory.

4.3 Contract-first vs code-first

- **Contract-first:** Write the OpenAPI / `.proto` / GraphQL SDL first, generate stubs and clients from it. The schema is the source of truth; reviewers review the *contract*. Preferred at scale (Google, Amazon).
- **Code-first:** Write handlers, generate the spec from annotations. Faster for small teams but the contract drifts from intent.

4.4 Idempotency, safety, cacheability (preview)

Three orthogonal properties that drive most REST decisions:

- **Safe** = no observable side effects (read-only). GET, HEAD.
- **Idempotent** = N identical calls have the same effect as one. GET, PUT, DELETE.
- **Cacheable** = the response can be stored and reused. GET responses with appropriate headers.

These are covered in depth below — they are the single most-tested API-design concept.

5 REST In Depth

REST (**Representational State Transfer**, Roy Fielding, 2000) is an *architectural style*, not a protocol. A system is "RESTful" if it obeys six constraints:

1. **Client–Server** — separate UI from storage; they evolve independently.
2. **Stateless** — no server-side session; every request self-contained.
3. **Cacheable** — responses explicitly mark themselves cacheable or not.
4. **Uniform Interface** — resources are identified by URIs and manipulated via a small, standard set of verbs; representations are self-descriptive; hypermedia drives state (HATEOAS).
5. **Layered System** — client can't tell if it's talking to the origin or a proxy/CDN/gateway.
6. **Code on Demand** (optional) — server may ship executable code (rarely used).

In practice "REST" colloquially means "JSON over HTTP using verbs and status codes sensibly." The constraints above are the ideal; the Richardson Maturity Model (below) measures how close you are.

5.1 HTTP Methods --- Full Semantics

The verb is *the operation*; the URL is *the resource*. Never put verbs in URLs (`POST /createUser` is wrong — use `POST /users`).

Method	Safe	Idempotent	Cacheable	Has body	Purpose
GET	Yes	Yes	Yes	No (ideally)	Retrieve a resource/collection
HEAD	Yes	Yes	Yes	No	Like GET, headers only (size, ETag, existence)
OPTIONS	Yes	Yes	No	No	Discover allowed methods; CORS preflight
POST	No	**No**	Rarely	Yes	Create a subordinate resource, or non-CRUD action
PUT	No	**Yes**	No	Yes	Replace a resource entirely (or create at a known URI)
PATCH	No	**No***	No	Yes	Partially update a resource
DELETE	No	**Yes**	No	No	Remove a resource

Why each idempotency value — this is the most-asked sub-question:

- **GET is safe & idempotent** because reads don't mutate state. Caches, prefetchers, and retries can fire GET freely. *Never* hide a mutation behind GET (e.g. `GET /users/42/delete`) — a crawler or browser prefetch will trigger it.
- **PUT is idempotent** because it *sets* the resource to a fully-specified state: `PUT /users/42 {name:"Alice", email:"a@x.com"}` ten times leaves the user identical to once. The result depends only on the payload, not on the current state.
- **DELETE is idempotent** because the *end state* (resource gone) is the same after one or ten calls. Subtlety: the *first* call returns `200/204`, later calls may return `404` — but the *resource state* is unchanged, which is the definition of idempotent. Some APIs return `204` every time to make retries simpler.
- **POST is NOT idempotent** because it creates a *new* subordinate resource each time. Two identical `POST /orders` calls create two orders → the classic double-charge bug on retry. This is exactly why idempotency keys exist (see below).
- **PATCH is NOT idempotent in general** (*). A JSON Merge Patch that *sets* fields can be idempotent, but a PATCH expressing a delta — e.g. `{"op":"increment","path":"/balance","value":10}`

— is not: applying it twice adds 20. Treat PATCH as non-idempotent unless you prove otherwise.

```http
GET    /users/42            → fetch user 42
PUT    /users/42            → replace user 42 (full representation in body)
PATCH  /users/42            → partial update (only changed fields in body)
DELETE /users/42            → delete user 42
POST   /users              → create a new user (server assigns id)
HEAD   /files/big.zip       → check size/existence without downloading
```

Scenario — retry storm: A mobile client times out after `POST /payments` but the server actually processed it. The client retries. Without protection, the user is charged twice. With GET/PUT/DELETE this never happens (idempotent by construction); with POST you *must* add an idempotency key. This single scenario justifies most of this guide.

5.2 HTTP Status Codes --- When To Use Each

Status codes are part of your contract. Clients branch on them. Use the *most specific* code that's correct.

Code	Name	Use it when…
200	OK	Successful GET/PUT/PATCH; or POST that returns data without creating an addressable resource
201	Created	POST/PUT created a new resource. **Include a `Location` header** pointing to it, and usually the body
202	Accepted	Request accepted for **async** processing; work not done yet. Return a status URL to poll
204	No Content	Success, nothing to return (e.g. DELETE, or PUT with no echo body)
301	Moved Permanently	Resource permanently at a new URL; clients/caches should update. Used for API URL migrations
304	Not Modified	Conditional GET matched `If-None-Match`/`If-Modified-Since` → client's cache is still valid, no body sent
400	Bad Request	Malformed syntax — unparseable JSON, missing required param, wrong type
401	Unauthorized	**Not authenticated** — no/invalid credentials. Send `WWW-Authenticate`. (Misnamed: means *unauthenticated*)
403	Forbidden	Authenticated but **not allowed** — valid token, insufficient scope/permission
404	Not Found	Resource doesn't exist (or you're hiding its existence from this caller — sometimes preferred over 403 to avoid leaking)
405	Method Not Allowed	URL exists but verb isn't supported (`DELETE /reports` where reports are read-only). Include `Allow` header
409	Conflict	Request conflicts with current state — duplicate unique key, edit conflict (optimistic concurrency via ETag), already-processed idempotency key with a different body
410	Gone	Resource *deliberately* removed and won't come back — used for deprecated/sunset API versions
422	Unprocessable Entity	Syntactically valid but **semantically invalid** — JSON parsed fine, but `email` isn't a valid email, or `quantity` is negative
429	Too Many Requests	Rate limit exceeded. **Include `Retry-After`** and `X-RateLimit-*`
500	Internal Server Error	Unexpected server bug. Never leak stack traces; return a request ID

Code	Name	Use it when…
503	Service Unavailable	Server temporarily down/overloaded/in maintenance. Include `Retry-After`. Signals "retry later," distinct from a permanent error

400 vs 422 (frequent follow-up): 400 = "I can't even parse this." 422 = "I parsed it, but the *content* breaks a business rule." Example: `{"age": "-5"}` where age must be a positive integer → JSON is valid, value isn't → 422. A truncated JSON body → 400. Many teams use 400 for both to keep it simple — but knowing the distinction signals depth.

401 vs 403 (the classic trap): 401 = *who are you?* (authenticate). 403 = *I know who you are, you still can't.* If a user with a read-only token tries `DELETE`, that's **403**, not 401.

409 in practice — optimistic concurrency:

```http
GET /docs/7
→ 200 OK
  ETag: "v12"

PUT /docs/7
If-Match: "v12"
{ "title": "New" }

# If someone else saved v13 in between:
→ 409 Conflict   (your edit is based on a stale version)
```

Scenario — async export: `POST /reports/export` kicks off a 5-minute job. Returning 200 with the data would force a 5-minute hung connection. Instead return **202 Accepted** with `Location: /reports/export/job-abc`; the client polls that URL (200 with status `pending`/`done`, eventually a download link). This is the standard *long-running operation* pattern.

5.3 Key HTTP Headers

```http
# Request
Content-Type: application/json          # format of the body you're sending
Accept: application/json                # formats you can handle in the response
Authorization: Bearer eyJhbGci...       # credentials
If-None-Match: "abc123"                 # conditional GET — only send body if ETag changed
If-Match: "abc123"                      # conditional write — fail if resource changed
Idempotency-Key: 8f3c...               # dedup key for unsafe retries (POST)

# Response
Content-Type: application/json; charset=utf-8
Location: /orders/42                     # where the newly created resource lives (201)
ETag: "abc123"                          # version tag of this representation
Cache-Control: max-age=3600, private    # how long/where this may be cached
Retry-After: 30                          # seconds to wait (429/503)
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 42
X-RateLimit-Reset: 1718500000           # epoch when the window resets
```

ETag + If-None-Match (conditional GET) — saves bandwidth:

```http
# First request
GET /products/9
→ 200 OK
  ETag: "p9-v4"
  { ...big payload... }

# Later — client revalidates
GET /products/9
If-None-Match: "p9-v4"
→ 304 Not Modified      # empty body; client reuses its cache
```

The ETag is a content fingerprint (hash or version number). `If-None-Match` lets the server answer "nothing changed, keep your copy" with a tiny 304 instead of re-sending the whole resource. `Cache-Control: max-age=3600` says "you may reuse without even asking for 3600s"; after that, the client *revalidates* with the ETag.

5.4 Content Negotiation

The client states preferences; the server picks the best representation it supports.

```http
GET /reports/9
Accept: application/json, application/xml;q=0.8, */*;q=0.1
Accept-Language: en-US, en;q=0.9
Accept-Encoding: gzip, br
```

The `q` values are quality weights (0–1). The server responds with `Content-Type: application/json` (its best match) and may `406 Not Acceptable` if it can't satisfy any. Same URL, multiple representations — this is the "Representational" in REST.

5.5 Richardson Maturity Model

A four-level ladder measuring how "RESTful" an API is:

```
Level 0 — The Swamp of POX (Plain Old XML)
          One URL, one verb (POST), RPC-over-HTTP.
          e.g. POST /api with {"method":"getUser","id":42}

Level 1 — Resources
          Many URLs, still one verb.
          POST /users/42, POST /users/42/delete

Level 2 — HTTP Verbs + Status Codes   ← where ~95% of "REST" APIs live
          GET/POST/PUT/DELETE used correctly, proper status codes.
          GET /users/42 → 200, DELETE /users/42 → 204

Level 3 — Hypermedia Controls (HATEOAS)
          Responses include links telling the client what it can do next.
```

HATEOAS (Hypermedia As The Engine Of Application State): the server returns not just data but the *links/actions* available from the current state, so the client doesn't hard-code URLs.

```json
{
  "id": "order_42",
  "status": "pending_payment",
```

```
  "total": 4999,
  "currency": "usd",
  "_links": {
    "self":   { "href": "/orders/42" },
    "pay":    { "href": "/orders/42/payment", "method": "POST" },
    "cancel": { "href": "/orders/42", "method": "DELETE" }
  }
}
```

When the order becomes `shipped`, the `pay` and `cancel` links disappear and a `track` link appears — the client navigates by *following links*, not by constructing URLs. **Reality check:** full HATEOAS is rare (it's verbose and most clients hard-code paths anyway), but interviewers love to ask about it. Know what it is, give the example, and note honestly that most production "REST" APIs stop at Level 2. Hypermedia *pagination links* (`next`/`prev`) are the one HATEOAS idea that's genuinely common.

5.6 Statelessness in REST

No `login → server stores session → subsequent requests assume session`. Instead each request carries a token. Benefits: any server handles any request (horizontal scaling), failures are trivial to recover (no lost session), and caching/proxying work because requests are self-describing. The cost: tokens must be re-validated each request (cheap with stateless JWTs) and the client carries more per-request weight.

6 Resource Modeling & URL Design

The art of REST is choosing the right **nouns**. Get the resource model right and the verbs fall out automatically.

6.1 Nouns, not verbs

Bad (verb in URL)	Good (noun + verb method)
`POST /createOrder`	`POST /orders`
`GET /getUserById?id=42`	`GET /users/42`
`POST /users/42/setName`	`PATCH /users/42`
`GET /listActiveUsers`	`GET /users?status=active`

Rules of thumb: **plural nouns** for collections (`/users` not `/user`), **lowercase kebab-case** paths, **IDs in the path** (`/users/42`), **no file extensions** (use `Accept` for format), **no trailing verbs**.

6.2 Collections and sub-resources

```http
/users                  collection
/users/42               single resource
/users/42/orders        sub-collection (orders belonging to user 42)
/users/42/orders/7      a specific order of that user
/orders/7               the same order, addressed globally (often also provided)
```

Nest only **one level deep** in general. `/users/42/orders/7/items/3/reviews` is a pain to build, document, and authorize — instead expose `/items/3/reviews` directly and let `/orders/7` link to its items. Deep nesting also couples resources that should be independently addressable.

6.3 Actions --- the exception to nouns

Some operations don't map cleanly to CRUD on a resource: *send* an email, *cancel* an order, *retry* a job, *publish* a post. Two accepted patterns:

1. **Action as a sub-resource (POST):** `POST /orders/42/cancellation` or `POST /orders/42:cancel` (Google's custom-method colon syntax). Treat the action as creating an event.
2. **State transition via PATCH:** `PATCH /orders/42 {"status":"cancelled"}` — cleaner when the action is "set a field," but you lose the ability to attach action-specific parameters cleanly.

Prefer modeling the *result* as a resource where possible (`POST /transfers` instead of `POST /accounts/1/sendMoney`). When you genuinely need an RPC-style action, document it and don't pretend it's pure REST.

6.4 Filtering, sorting, field selection --- query params

Query params modify a *collection read*; they never identify the resource.

```http
GET /products?category=books&price_gte=10&price_lte=50      # filtering
GET /products?sort=-price,name                               # sort by price desc, then name asc
GET /products?fields=id,name,price                           # sparse fieldset (reduce payload)
GET /products?q=harry+potter                                 # full-text search
GET /products?category=books&page_size=20&cursor=eyJ...      # filter + paginate
```

Conventions worth knowing: `-` prefix for descending sort; `_gte`/`_lte`/`_ne` suffix operators (or OData `$filter`); `fields=` (sparse fieldsets, the REST answer to GraphQL over-fetching). **Filters narrow a collection; field selection narrows each item.** Keep these orthogonal.

6.5 Bulk operations

Single-resource endpoints don't scale when a client must create/update 10,000 rows (10,000 round trips). Options:

- **Batch endpoint:** `POST /products/batch` with an array body, returning a **per-item status array** (some succeed, some fail → typically `200` overall with individual results, or `207 Multi-Status`):

```json
{
  "results": [
    { "index": 0, "status": 201, "id": "p_1" },
    { "index": 1, "status": 422, "error": { "code": "invalid_price" } }
  ]
}
```

- **Async bulk import:** for huge volumes, `POST /imports` returns `202` + a job URL; the client uploads a file and polls.

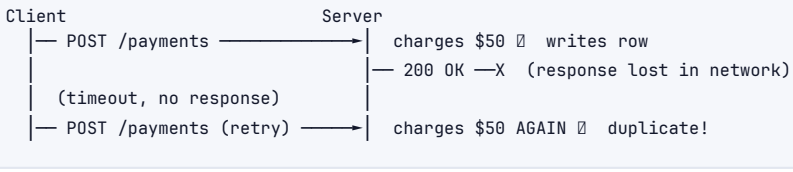
Interview takeaway: mention that bulk endpoints need a *partial-failure* contract — what happens when item 7 of 100 fails? Returning a single 400 for the whole batch is usually wrong; clients need to know *which* items failed so they can retry just those.

7 Idempotency & Reliability

> This is, with pagination, the highest-value API-design topic. Master the mechanism, not just the word.

Definition: an operation is idempotent if performing it N times produces the same *server state* as performing it once. It's the property that makes **safe retries** possible — and networks *force* retries (timeouts, dropped ACKs, load-balancer hiccups).

7.1 The problem



GET/PUT/DELETE are idempotent by construction. **POST is not** — and POST is exactly what you use for "create order," "charge card," "send message." So we add idempotency *manually*.

7.2 Idempotency keys --- the mechanism

The client generates a unique key (UUID) per *logical* operation and sends it with every retry of that operation:

```http
POST /payments
Idempotency-Key: 9f1c2e7a-3b6d-4e8f-a1c2-7d9e0f1a2b3c
Content-Type: application/json

{ "amount": 5000, "currency": "usd", "source": "card_xyz" }
```

The server keeps a **dedup table** keyed by `(api_key, idempotency_key)`:

```sql
CREATE TABLE idempotency_keys (
  api_key          TEXT        NOT NULL,
  idempotency_key  TEXT        NOT NULL,
  request_hash     TEXT        NOT NULL,   -- hash of the request body
  response_status  INT,                    -- cached response (NULL until done)
  response_body    JSONB,
  state            TEXT        NOT NULL,    -- 'in_progress' | 'completed'
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (api_key, idempotency_key)
);
```

Server algorithm:

```
1. Atomically INSERT (api_key, key, request_hash, state='in_progress').
   - On primary-key conflict → this key was seen before:
       a. If stored request_hash ≠ this request's hash → 422/409
          ("key reused with a different body" — client bug, refuse).
       b. If state='completed'   → return the stored response verbatim.
       c. If state='in_progress' → original request is still running →
```

```
          return 409 Conflict (or block briefly and retry).
2. (We won the insert → first time.) Execute the real work (charge card)
   inside the SAME transaction or with careful ordering.
3. UPDATE the row: state='completed', store response_status + response_body.
4. Return the response.
```

The `INSERT ... ON CONFLICT` (or a unique constraint catch) is the atomic gate that guarantees *exactly one* execution even under concurrent retries. The stored response makes retries return the *identical* result (same order ID, same charge), which is what "exactly-once *effects*" means in practice: the network may deliver the request many times (at-least-once delivery), but the *effect* happens once.

Key design choices:

- **Scope the key per client** (`api_key + idempotency_key`) so two customers can't collide.
- **Expire keys** (e.g. 24h TTL) — they're only needed during the retry window; storing forever is wasteful.
- **Hash the body** so a client reusing a key for a *different* request is caught (that's a bug, not a retry).
- **What's idempotent is the *effect*, not the response code** — a retry of a successful create may legitimately return `200` instead of the original `201`.

7.3 Idempotent PUT vs non-idempotent PATCH

```Http
PUT /users/42  { "name":"Alice", "tier":"gold" }     # idempotent: sets full state
PATCH /users/42  { "op":"add", "path":"/credits", "value":10 }   # NOT idempotent: +10 each call
```

Prefer **absolute** updates (`set credits = 100`) over **relative** ones (`add 10 credits`) when you want idempotency for free. If you must express a delta, protect it with an idempotency key or an ETag (`If-Match`) so a retried delta isn't applied twice.

7.4 Reliability checklist (mention in interviews)

- **Idempotency keys** for all unsafe creates/charges.
- **Retries with exponential backoff + jitter** on the client; honor `Retry-After`.
- **Timeouts** on every call (no infinite hangs).
- **Circuit breakers** to stop hammering a failing dependency.
- **Outbox pattern** when an API call must atomically also publish an event (write the event to a DB outbox in the same transaction, relay it asynchronously).

Interview takeaway: "Networks give you *at-least-once* delivery; idempotency turns that into *exactly-once effects*." Sketch the dedup table and the atomic insert — that concrete mechanism is what separates a strong answer.

8 Pagination

Returning a 50-million-row collection in one response is impossible. Pagination splits it into pages. The choice of *scheme* has deep consequences for correctness and performance.

8.1 Offset / Limit

```http
GET /events?limit=20&offset=40        # "skip 40, take 20" → page 3
```

```sql
SELECT * FROM events
ORDER BY created_at DESC
LIMIT 20 OFFSET 40;
```

Why it degrades at large offsets: `OFFSET 1000000` forces the database to *scan and discard* the first million rows before returning 20. Cost is **O(offset)** — page 1 is instant, page 50,000 times out. There's no index trick that fixes a pure offset scan.

Why it's unstable under writes: offset addresses a *position*, not an *item*. If a new event is inserted at the top while a user pages, every subsequent page **shifts by one** → the user sees a duplicate row, or skips one entirely.

```
t0: [A B C D E]   user reads page1 = [A B]   (offset 0, limit 2)
t1: insert Z at top → [Z A B C D E]
t2: user reads page2 (offset 2, limit 2) = [B C]   ← B seen twice, A's neighbor skipped
```

Offset is fine for *small, bounded, mostly-static* datasets and admin UIs that need "jump to page 7." It's wrong for large, fast-changing feeds.

8.2 Cursor / Keyset Pagination

Instead of "skip N," remember the *last item you saw* and ask for "items after this one." The cursor encodes the sort key of the last row.

```sql
-- Page 1
SELECT * FROM events
ORDER BY created_at DESC, id DESC
LIMIT 20;

-- Next page: pass the (created_at, id) of the last row as a cursor.
-- "Give me rows strictly before that point" — a row comparison, index-friendly:
SELECT * FROM events
WHERE (created_at, id) < ('2026-06-15T10:00:00Z', 91234)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

The `WHERE (created_at, id) < (...)` uses the index to **seek directly** to the boundary — cost is **O(limit)**, *independent of how deep you are*. Page 50,000 is as fast as page 1. Including `id` as a tiebreaker makes the order **total** (no ambiguity when two rows share `created_at`), which is essential for correctness.

The cursor is opaque — base64-encode it so clients treat it as a black box and you can change its internals later:

```python
cursor = base64({"created_at":"2026-06-15T10:00:00Z","id":91234})
       = "eyJjcmVhdGVkX2F0IjoiMjAyNi0wNi0xNVQxMDowMDowMFoiLCJpZCI6OTEyMzR9"
```

```http
GET /events?limit=20
→ 200 OK
{
  "data": [ ... ],
  "page": {
    "next_cursor": "eyJjcmVhdGVVk...",
    "has_more": true
  }
}

GET /events?limit=20&cursor=eyJjcmVhdGVVk...
```

Stable under writes: new rows inserted at the top don't shift the cursor's anchor, so no duplicates or skips while paging forward. The trade-off: **no random access** — you can't "jump to page 500," only walk next/prev. For infinite-scroll feeds (the common case at scale) that's exactly what you want.

8.3 Page tokens (Google-style)

A generalization of cursors: the server returns an opaque `next_page_token`; the client passes it back. The token can encode a cursor, an offset, or even a server-side scroll/snapshot ID (e.g. Elasticsearch `search_after`/scroll). Clients never parse it. This is the most flexible scheme and what most large APIs converge on.

8.4 Comparison

	Offset/Limit	Cursor/Keyset	Page token
Deep-page performance	O(offset) — degrades badly	O(limit) — constant	O(limit) (impl-dependent)
Stable under inserts	No (dup/skip)	Yes	Yes
Random access ("page 7")	Yes	No	No
Total count available	Easy	Hard/expensive	Hard
Client simplicity	Simplest	Medium (opaque cursor)	Medium
Best for	Small, static, admin tables	Large feeds, infinite scroll	Public APIs at scale

Interview takeaway: "For a feed at scale I'd use **cursor (keyset) pagination** — it's O(page size) regardless of depth and stable under concurrent writes, unlike offset which scans-and-discards and shifts pages when rows are inserted." Then write the `WHERE (created_at, id) < (...)` query. That sentence + that query is a near-guaranteed signal.

9 Versioning & Evolution

APIs are *forever* once a client depends on them. The goal: **add capability without breaking existing integrations**, and when you must break, do it loudly and slowly.

9.1 Where to put the version

Strategy	Example	Pros	Cons
URL path	`GET /v2/users/42`	Obvious, easy to route/cache, trivially testable in a browser	"Version" pollutes every URL; encourages big-bang v2
Custom header	`X-API-Version: 2`	Clean URLs; resource identity unchanged across versions	Invisible in logs/browser; easy to forget; caching by header is fiddly
Media type (content negotiation)	`Accept: application/vnd.acme.v2+json`	Most "RESTful" (versions the *representation*)	Verbose, poor tooling support, confuses many developers
Date-based (Stripe)	`Stripe-Version: 2024-04-10`	Fine-grained, pin clients to a snapshot of behavior	Server must maintain transformation layers per date

URL path versioning is the pragmatic default — it's the easiest to understand, route, and debug. Reserve a real `/v2` for *breaking* changes only; do everything else additively within `/v1`.

9.2 Additive (non-breaking) vs breaking changes

Non-breaking — safe to ship to existing clients (no version bump):

- Adding a **new optional** field to a response (clients ignore unknown fields).
- Adding a **new endpoint** or a new optional query parameter.
- Adding a new value to an enum **only if** clients are documented to tolerate unknowns.
- Relaxing a validation rule (accepting more inputs).

Breaking — requires a new version (or never do it):

- **Removing or renaming** a field (`fullName` → `name`).
- **Changing a type** (`amount: "5.00"` string → `5.00` number, or cents vs dollars).
- Making an **optional field required**, or adding a new required request field.
- Changing **default behavior**, error codes, or the meaning of an existing field.
- Tightening validation so previously-valid requests now fail.

```json
// v1 response
{ "id": 42, "name": "Alice" }

// SAFE evolution — added optional field, old clients ignore it
{ "id": 42, "name": "Alice", "avatar_url": "https://..." }

// BREAKING — renamed field; v1 clients reading `name` now get null → needs /v2
{ "id": 42, "full_name": "Alice" }
```

The robustness principle (Postel's Law): *be conservative in what you send, liberal in what you accept.* Concretely: clients must **ignore unknown fields** (so the server can add them freely), and servers should accept extra/unknown request fields gracefully where reasonable. This is what makes additive evolution possible.

9.3 Deprecation process

1. **Announce** in docs + changelog + email to API consumers; set a sunset date (e.g. 6–12 months out).
2. **Signal in responses:** `Deprecation: true` and `Sunset: Wed, 01 Jul 2026 00:00:00 GMT` headers, plus a `Warning` header.

3. **Monitor** which clients still call the old version (by API key) and reach out to laggards.
4. **Sunset:** after the date, return `410 Gone` (deliberately removed) rather than 404.

```Http
GET /v1/users/42
→ 200 OK
  Deprecation: true
  Sunset: Wed, 01 Jul 2026 00:00:00 GMT
  Link: <https://docs.acme.com/migrate-v2>; rel="deprecation"
```

9.4 Migration scenario

You ship `amount` as **dollars (float)** in v1 and realize floats cause rounding bugs; you want **cents (integer)**.

- *Wrong:* silently change v1's `amount` to integer cents → every existing client now displays `$5000` instead of `$50`. Catastrophic.
- *Right:* add `amount_cents` (integer) as a **new optional field** in v1 alongside the old `amount`. Document `amount` as deprecated. Move all-new clients to `amount_cents`. In `/v2`, drop `amount` entirely. Existing v1 clients keep working untouched.

Interview takeaway: "I version only on *breaking* changes and ship everything else additively, relying on clients ignoring unknown fields. URL versioning by default, with a real deprecation policy: `Deprecation`/`Sunset` headers, a 6-month window, then `410 Gone`." That shows you understand evolution is a *process*, not a header.

10 Authentication & Authorization

AuthN = *who are you?* **AuthZ** = *what are you allowed to do?* Two distinct steps; conflating them (`401` vs `403`) is a classic mistake.

10.1 API Keys

A long random secret string identifying the *application* (not a user).

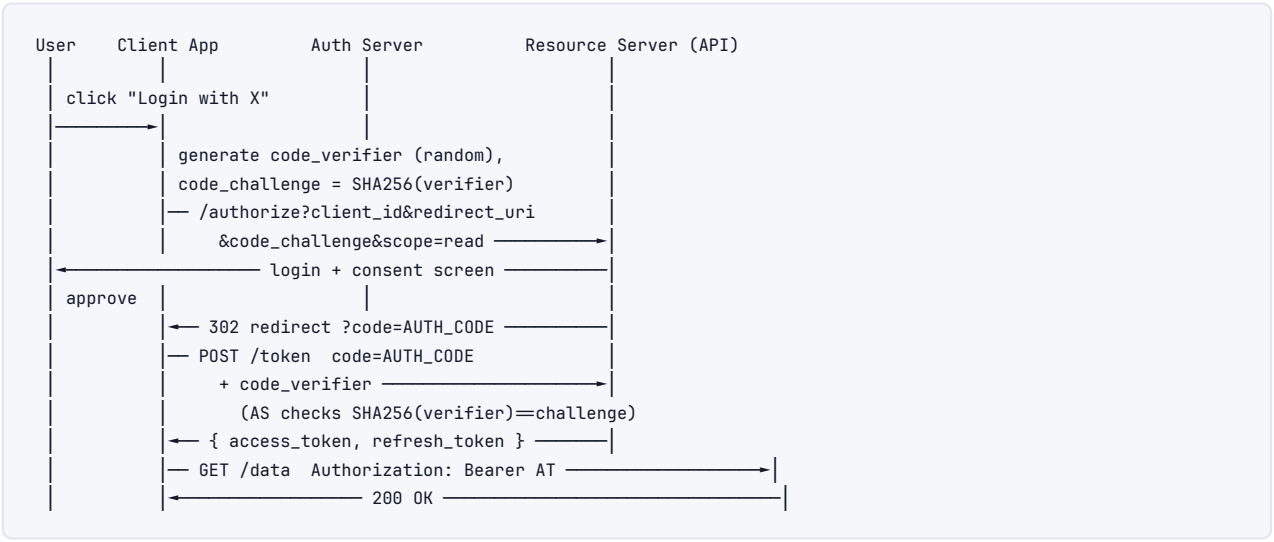
```Http
GET /data
Authorization: Bearer sk_live_4eC39Hq...    # or:  X-API-Key: sk_live_...
```

- **Pros:** dead simple, great for server-to-server and getting started.
- **Cons:** static, long-lived, no built-in user identity or granular scopes, hard to rotate, disastrous if leaked (treat like a password — never in URLs/logs/client-side code). Mitigate with key prefixes (`sk_live_` vs `pk_test_`), hashing at rest, per-key rate limits, and rotation.

10.2 OAuth 2.0

The standard for **delegated** authorization: let app A act on a user's behalf at service B *without* A ever seeing the user's password. Four roles: **Resource Owner** (user), **Client** (the app), **Authorization Server** (issues tokens), **Resource Server** (the API).

Authorization Code flow + PKCE (for web/mobile/SPA — the modern default):



Why PKCE? Without it, an attacker who intercepts the redirect (`?code=...`) on a mobile device could exchange the code for a token. PKCE binds the code to a one-time secret (`code_verifier`) that only the legitimate client knows — the stolen code is useless without it. PKCE is now recommended for *all* clients, not just public ones.

Client Credentials flow (machine-to-machine, no user):

```http
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=svc_orders
&client_secret=...
&scope=inventory.read inventory.write
→ { "access_token": "...", "expires_in": 3600, "token_type": "Bearer" }
```

Used for service-to-service: there's no user, the *service itself* is the principal. Other flows you should be able to name: **Refresh Token** (swap a long-lived refresh token for a fresh short-lived access token without re-login), and the **deprecated** Implicit and Resource Owner Password flows (don't recommend them — Implicit leaks tokens in URLs; Password requires giving the app your credentials).

10.3 JWT (JSON Web Token)

A **self-contained, signed** token. The resource server can validate it *without* a database lookup — that's what makes it great for stateless, horizontally-scaled APIs.

```
header.payload.signature   (three base64url parts joined by dots)

eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9 . eyJzdWIiOiJ1c2VyXzQyIiwiZXhwIjoxNzE4NX0 . SflKxw...
```

```json
// header
{ "alg": "RS256", "typ": "JWT" }
// payload (claims)
{
  "iss": "https://auth.acme.com",   // issuer
```

```
  "sub": "user_42",                  // subject (who)
  "aud": "api.acme.com",             // audience (who it's for)
  "exp": 1718500000,                 // expiry (epoch) — REQUIRED, keep short
  "iat": 1718496400,                 // issued-at
  "scope": "orders.read orders.write"
}
// signature = RS256(base64(header) + "." + base64(payload), private_key)
```

The server verifies the signature with the issuer's **public key** (RS256/asymmetric) — so any service can validate without holding a secret. **Critical: the payload is base64, not encrypted — anyone can read it. Never put secrets in a JWT.**

JWT pitfalls (high-value follow-ups):

- **Revocation is hard.** A JWT is valid until `exp` no matter what — you can't "log out" a stateless token server-side. Mitigations: **short expiry** (5–15 min) + refresh tokens; a **denylist** of revoked token IDs (`jti`) in Redis (reintroduces a lookup, partially defeating the point); or token versioning per user.
- **`alg: none` attack** — early libraries accepted a token claiming `"alg":"none"` (unsigned). Always pin the expected algorithm server-side; never trust the header's `alg`.
- **Key confusion (RS256→HS256)** — attacker switches algorithm so your RSA *public* key is used as an HMAC *secret*. Pin the algorithm.
- **Clock skew** — validate `exp`/`nbf` with a small leeway; ensure servers sync time (NTP).
- **Size** — JWTs ride in every request header; fat claims bloat every call.

Session tokens vs JWTs: an opaque session ID + server-side store gives instant revocation but needs a lookup per request (or a shared cache). JWTs give stateless validation but weak revocation. Many systems use **short-lived JWTs + a refresh token** to get most of both.

10.4 mTLS (mutual TLS)

Both client *and* server present certificates; each verifies the other. Used for **service-to-service** auth inside a mesh (Istio/Linkerd, SPIFFE/SPIRE issuing short-lived certs). Stronger than bearer tokens (the credential is bound to the TLS connection and can't be replayed elsewhere), but heavier to operate (cert issuance/rotation).

10.5 Scopes

Scopes are **coarse-grained permissions** attached to a token, limiting what it can do (`read:orders`, `write:payments`). They implement **least privilege**: a read-only dashboard gets a token with only `read:*` scopes, so even if leaked it can't mutate. The resource server checks the required scope per endpoint → returns **403** if missing. Scopes ≠ roles: scopes bound the *token*; roles/RBAC/ABAC decide the *user's* permissions. Real systems combine both.

Interview takeaway: "AuthN before AuthZ. For users, OAuth2 Authorization-Code + PKCE issuing short-lived JWTs; for services, client-credentials or mTLS. JWTs are stateless and fast to validate but hard to revoke — so I keep `exp` short and pair with refresh tokens. Scopes enforce least privilege; missing scope is 403, not 401."

11 Rate Limiting & Throttling

Rate limiting protects the API from **abuse, accidental floods, and noisy neighbors**, and enforces fair use / pricing tiers. It's both a reliability and a business mechanism.

11.1 Algorithms

Token Bucket — a bucket holds up to `B` tokens; tokens refill at rate `r`/sec; each request spends one. Empty bucket → reject (429).

```
refill r=10/sec, capacity B=20
A burst of 20 requests passes instantly (drains the bucket),
then throughput is capped at the steady 10/sec refill.
→ allows bursts up to B, smooths the average to r.
```

```python
# Redis-backed token bucket (atomic via Lua in practice)
# stored per key: tokens, last_refill_ts
now = time.time()
elapsed = now - last_refill
tokens = min(B, tokens + elapsed * r)     # refill
if tokens ⩾ 1:
    tokens -= 1
    allow()                                # 200
else:
    reject()                               # 429, Retry-After = (1 - tokens)/r
last_refill = now
```

Leaky Bucket — requests queue and *drain at a fixed rate*. Overflow is dropped. Output is perfectly smooth (constant rate), but it allows **no bursts** — even a momentary spike past the drain rate is shaped or dropped. Good for protecting a downstream that needs steady input.

Fixed Window Counter — count requests per fixed clock window (e.g. per minute), reset at the boundary.

```
INCR ratelimit:user42:minute_1718500   # atomic counter
EXPIRE ratelimit:user42:minute_1718500 60
if count > limit: reject (429)
```

Simple and cheap, but the **boundary burst problem**: with a limit of 100/min, a client can send 100 at `12:00:59` and 100 more at `12:01:00` → **200 requests in 2 seconds**, double the intended rate, because the counter reset at the window edge.

Sliding Window Log — store a timestamp per request in a sorted set; count entries in the trailing window. Exact, no boundary burst, but **O(requests) memory**.

```python
ZREMRANGEBYSCORE key 0 (now - 60000)   # evict entries older than 60s
ZADD key now now                       # record this request
count = ZCARD key
if count > limit: reject
```

Sliding Window Counter — the production sweet spot: approximate the sliding window by weighting the previous fixed window's count by how much of it overlaps the current sliding window.

```
limit = 100/min
prev_window_count = 80, curr_window_count = 30
elapsed_in_current = 15s  → 25% into the current minute → 75% of prev window still in view
estimate = prev*0.75 + curr = 80*0.75 + 30 = 90  →  under 100, allow
```

O(1) memory, smooths the boundary burst, accurate enough for almost everyone.

Algorithm	Bursts	Memory	Accuracy	Notes
Token bucket	Allowed (up to B)	O(1)	Good	Most common; natural ”burst + steady rate”
Leaky bucket	None (shaped)	O(1) + queue	Good	Smooth output; protects fragile downstreams
Fixed window	2× at edges	O(1)	Poor at edges	Simplest
Sliding log	None	O(n)	Exact	Memory-heavy
Sliding counter	Minimal	O(1)	Very good	Best general default

11.2 The 429 response

```http
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1718500060
Content-Type: application/json

{ "error": { "code": "rate_limited", "message": "Too many requests, retry in 30s." } }
```

Always return `Retry-After` (seconds or HTTP-date) so well-behaved clients back off correctly, plus `X-RateLimit-*` so clients can *self-throttle* before hitting the wall. A good API makes the limits *visible*, not just enforced.

11.3 Dimensions & tiers

Rate-limit by **API key**, **user ID**, **IP** (for anonymous/abuse), or **endpoint** (expensive endpoints get tighter limits). Offer **tiered limits** by plan (free: 100/min, pro: 10k/min) — this is how usage-based pricing is enforced. In a distributed deployment, the counter must be **shared** (Redis) so limits hold across all gateway instances; a per-instance counter lets a client get `N × instances`.

Interview takeaway: ”Token bucket for burst-tolerant user APIs, leaky bucket to protect a fragile downstream, sliding-window-counter as a memory-efficient accurate default. Counters live in Redis so they're global across gateway nodes. Always emit `Retry-After` + `X-RateLimit-*`.”

12 Error Handling

Errors are part of the contract. Inconsistent or vague errors are the #1 complaint about real APIs. Three rules: **a consistent envelope**, **a stable machine-readable code**, and **enough detail to act** without leaking internals.

12.1 Consistent error envelope

```json
{
  "error": {
    "code": "insufficient_funds",          // stable, machine-readable — clients branch on this
    "message": "Your card has insufficient funds.",    // human-readable, safe to surface
    "request_id": "req_8f3c2e7a",          // for support / log correlation
    "doc_url": "https://docs.acme.com/errors/insufficient_funds",
    "details": [
      { "field": "amount", "issue": "exceeds_balance" }
    ]
  }
}
```

The **`code`** is what clients switch on — never make clients parse the human `message` (it changes, gets localized). The HTTP status gives the *category* (4xx client, 5xx server); the `code` gives the *specifics*. Always include a **`request_id`** that also appears in your logs so a developer can paste it into a support ticket and you can find the exact failure.

12.2 RFC 7807 --- Problem Details for HTTP APIs

A standardized error format (`Content-Type: application/problem+json`):

```http
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "https://acme.com/probs/validation",   // URI identifying the problem type
  "title": "Validation failed",                    // short human summary
  "status": 422,
  "detail": "The 'email' field is not a valid email address.",
  "instance": "/users",                            // the specific occurrence
  "errors": [ { "field": "email", "code": "invalid_format" } ]
}
```

Worth knowing by name — Microsoft/Azure and Spring use it by default. Even if you roll your own envelope, mention RFC 7807 to show awareness of the standard.

12.3 Validation errors --- report them all at once

Don't fail on the first bad field; collect and return all violations so the user fixes them in one round trip:

```json
{
  "error": {
    "code": "validation_error",
    "message": "Request validation failed.",
    "details": [
      { "field": "email",    "code": "invalid_format" },
      { "field": "age",      "code": "must_be_positive" },
      { "field": "country",  "code": "required" }
    ]
  }
}
```

12.4 Partial failures

For batch endpoints and aggregations, a single status can't describe "3 of 100 failed." Return per-item results (or `207 Multi-Status`) so the client knows exactly what to retry:

```json
{
  "results": [
    { "id": "a", "status": "ok" },
    { "id": "b", "status": "error", "error": { "code": "not_found" } }
  ]
}
```

12.5 Don't leak internals

`500` must **never** return a stack trace, SQL, or internal hostnames — that's an information-disclosure vulnerability. Log the detail server-side keyed by `request_id`; return a generic message + that ID to the client.

Interview takeaway: consistent envelope + stable `code` + `request_id`, status for category and code for specifics, all validation errors at once, never leak internals, and name-drop RFC 7807.

13 gRPC In Depth

gRPC is Google's high-performance RPC framework: **Protocol Buffers** for the schema/serialization, **HTTP/2** for transport. You define a service in a `.proto` file, generate strongly-typed client + server code in any language, and call remote methods as if they were local functions.

13.1 Protobuf schema

```protobuf
syntax = "proto3";
package orders.v1;

service OrderService {
  rpc GetOrder   (GetOrderRequest)  returns (Order);                // unary
  rpc ListOrders (ListOrdersRequest) returns (stream Order);        // server stream
  rpc UploadEvents (stream Event)   returns (UploadSummary);        // client stream
  rpc Chat       (stream Message)   returns (stream Message);       // bidirectional
}

message GetOrderRequest {
  string order_id = 1;            // field NUMBERS (not names) are the wire identity
}

message Order {
  string order_id    = 1;
  string customer_id = 2;
  int64  amount_cents = 3;
  Status status      = 4;
  // field 5 reserved for future use — add new fields with NEW numbers, never reuse
}

enum Status {
  STATUS_UNSPECIFIED = 0;   // proto3 requires a zero default
  PENDING = 1;
  SHIPPED = 2;
```

```
27 }
```

Why field numbers matter: Protobuf encodes `field_number + type` on the wire, not field *names*. So you can **rename** a field freely (names are local) and **add** fields (new numbers) without breaking old clients — they ignore unknown numbers. The rules: never change a field's number, never reuse a deleted field's number (mark it `reserved`), and only widen types compatibly. This gives Protobuf its famous backward/forward compatibility — the gRPC equivalent of "ignore unknown JSON fields."

13.2 The four RPC types

```
Unary:          client ──req──►  ◄──resp── server     (like REST)
Server stream:  client ──req──►  ◄─r─r─r─r─ server     (download feed, large list)
Client stream:  client ─m─m─m─►  ◄──resp── server     (upload, telemetry batch)
Bidirectional:  client ◄─m─m─m─► server               (chat, live collaboration)
```

All four ride over a **single HTTP/2 connection**, multiplexed as independent streams.

13.3 Why HTTP/2 + Protobuf wins for internal services

- **Multiplexing:** many concurrent RPCs over one TCP connection (no connection-per-call, no HTTP/1.1 head-of-line blocking at the HTTP layer).
- **Binary Protobuf** is ~3–10× smaller than JSON and ~5–10× faster to (de)serialize — meaningful when service A calls service B millions of times/sec.
- **Streaming** is first-class (REST needs SSE/WebSocket bolt-ons).
- **Strong typing + codegen:** the `.proto` is the contract; clients/servers can't disagree on shape; refactors are compiler-checked.
- **Header compression (HPACK)** and persistent connections cut per-call overhead.

13.4 Code generation

`protoc` (+ language plugins) turns the `.proto` into: a **client stub** (you call `client.GetOrder(req)`), a **server interface** to implement, and all the serialization glue. The schema is the single source of truth shared across teams and languages.

13.5 When to use gRPC --- and the grpc-web limitation

Use gRPC when…	Use REST when…
Internal **service-to-service** calls	**Public/partner** APIs (universal tooling)
Latency/throughput critical	Human-debuggable JSON matters
You need streaming	Browsers are first-class clients
Polyglot services sharing a contract	Caching via HTTP infra is important

The browser limitation: browsers can't speak raw gRPC (no access to HTTP/2 framing / trailers from JS). You need **grpc-web** + a proxy (Envoy) that translates between the browser and the gRPC backend, and even then bidirectional streaming is limited. This is the single biggest reason gRPC stays *internal* and REST/GraphQL faces the public web.

Interview takeaway: "gRPC = Protobuf + HTTP/2: typed, binary, streaming, fast — ideal for internal microservices. It doesn't work natively in browsers (needs grpc-web + Envoy), so

public-facing surfaces stay REST/GraphQL. Protobuf field numbers give backward compatibility the way 'ignore unknown JSON fields' does for REST."

14 GraphQL In Depth

GraphQL (Meta, 2015) is a query language for APIs: the client sends a query specifying **exactly which fields it wants**, across multiple "resources," and gets back **exactly that shape** in one round trip. It solves REST's **over-fetching** (getting fields you don't need) and **under-fetching** (the N+1 round-trip problem where a screen needs data from many endpoints).

14.1 Schema + query

GraphQL

```graphql
# Schema (SDL) — the typed contract
type User {
  id: ID!
  name: String!
  orders(first: Int): [Order!]!
}
type Order {
  id: ID!
  total: Int!
  items: [Item!]!
}
type Query {
  user(id: ID!): User
}
```

GraphQL

```graphql
# Client query — one request, picks exactly what it needs
query {
  user(id: "42") {
    name
    orders(first: 5) {
      total
      items { name }
    }
  }
}
```

JSON

```json
// Response mirrors the query shape exactly
{ "data": { "user": { "name": "Alice",
  "orders": [ { "total": 4999, "items": [ { "name": "Book" } ] } ] } } }
```

One endpoint (`POST /graphql`), one round trip, no over/under-fetching. A mobile screen that needs a user, their last 5 orders, and each order's items would be 1 GraphQL call vs several REST calls.

14.2 Resolvers

Each field has a **resolver** — a function that fetches its data. `Query.user` resolves the user; `User.orders` resolves that user's orders; `Order.items` resolves each order's items. The engine walks the query tree, calling resolvers. This composition is the power *and* the danger.

14.3 The N+1 problem + DataLoader

Naive resolvers cause **N+1 queries**: resolving `user.orders` (1 query) then calling `Order.items` *once per order* (N queries) → 1 + N database hits.

```
Query 1:  SELECT * FROM orders WHERE user_id = 42        -- gets 5 orders
Query 2:  SELECT * FROM items WHERE order_id = 101        -- per order!
Query 3:  SELECT * FROM items WHERE order_id = 102
...       (5 separate item queries → N+1)
```

DataLoader fixes this by **batching + caching within a request**. Instead of firing a query per item-fetch, it collects all the keys requested in one tick of the event loop and issues a single batched query:

JavaScript

```
const itemLoader = new DataLoader(async (orderIds) => {
  // orderIds = [101, 102, 103, 104, 105]  — collected in one tick
  const rows = await db.query(
    'SELECT * FROM items WHERE order_id = ANY($1)', [orderIds]   // ONE query
  );
  // return items grouped per orderId, in the same order as the keys
  return orderIds.map(id => rows.filter(r => r.order_id === id));
});

// resolver:
Order.items = (order) => itemLoader.load(order.id);   // batched behind the scenes
```

Result: 1 query for orders + **1** batched query for all items = 2 queries instead of 6. DataLoader also **dedupes** repeated keys within the request. This is *the* GraphQL interview question.

14.4 Query depth / complexity limiting

Because clients control the query shape, a malicious or careless query can be ruinously expensive:

GraphQL

```
query { user(id:"1"){ orders{ items{ order{ user{ orders{ items{ ... }}}}}}}}
```

Defenses (all worth naming):

- **Max depth limiting** — reject queries nested beyond, say, 10 levels.
- **Complexity scoring** — assign each field a cost, multiply by list sizes (`first: 1000`), reject above a budget.
- **Persisted queries** — clients register approved queries ahead of time and send only a hash at runtime; the server runs only known-good queries (also shrinks the request and enables CDN caching). Used by Meta/Apollo in production.
- **Pagination caps** + timeouts + per-client rate limits.

14.5 Caching is harder than REST

REST caches beautifully: every GET URL is a cache key, HTTP infra (CDNs, browsers, `Cache-Control`/`ETag`) just works. GraphQL uses **one POST endpoint**, so HTTP-level caching doesn't apply out of the box. You cache *client-side* by object identity (Apollo/Relay normalized cache) and *server-side* per-resolver — more work. Persisted queries (with `GET`) restore some CDN cacheability.

14.6 When GraphQL vs REST

Prefer GraphQL	Prefer REST
Many diverse clients (mobile/web/TV) with different field needs	Simple CRUD, uniform clients
Deeply nested/related data fetched in one trip	HTTP caching / CDN is a priority
Rapidly evolving front-ends; want versionless evolution	Public API with broad tooling expectations
Want to avoid endpoint sprawl	File up/downloads, streaming

Interview takeaway: "GraphQL lets the client pick fields → kills over/under-fetching, great for mobile and diverse clients. The catch: the **N+1 problem** (fix with **DataLoader** batching), **query-cost** abuse (depth/complexity limits + persisted queries), and **caching** is harder than REST because it's one POST endpoint. REST stays better when HTTP caching and simplicity matter."

15 Webhooks & Async APIs

A webhook **inverts the direction**: instead of the client polling your API for changes, *you* call the client's URL when an event happens (`payment.succeeded`, `order.shipped`). It's "don't call us, we'll call you" — push instead of poll.

15.1 Delivery

```http
POST https://customer.com/webhooks/acme    # the URL the customer registered
Content-Type: application/json
X-Acme-Signature: t=1718500000,v1=5257a869e7...
X-Acme-Event-Id: evt_8f3c2e7a

{ "id": "evt_8f3c2e7a", "type": "payment.succeeded",
  "data": { "payment_id": "pay_42", "amount": 5000 } }
```

The consumer must respond **2xx quickly** (ideally just enqueue the event and return — do the heavy work async). A slow consumer causes the sender's delivery workers to back up.

15.2 HMAC signature verification

Webhooks arrive at a *public* URL — anyone could POST a fake `payment.succeeded`. So the sender signs each payload with a **shared secret** using HMAC; the consumer recomputes and compares:

```python
# Consumer side
import hmac, hashlib, time

def verify(payload_bytes, header, secret):
    # header: "t=1718500000,v1=5257a869e7..."
    parts = dict(p.split("=") for p in header.split(","))
    timestamp, sig = parts["t"], parts["v1"]
    signed = f"{timestamp}.".encode() + payload_bytes
    expected = hmac.new(secret.encode(), signed, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(expected, sig):     # constant-time compare!
        raise Exception("invalid signature")        # forged or tampered
    if time.time() - int(timestamp) > 300:          # reject old → replay defense
        raise Exception("timestamp too old")
```

Key points: include the **timestamp in the signed string** (so an attacker can't replay an old captured-but-valid payload forever), use a **constant-time** comparison (avoid timing attacks), and verify against the **raw body** (re-serializing JSON changes bytes and breaks the signature).

15.3 Retries

Networks fail and consumers go down, so senders **retry with exponential backoff** (e.g. immediately, then 1m, 5m, 30m, 2h... over ~24h). A webhook is delivered **at-least-once** — which means duplicates *will* happen.

15.4 Idempotent consumers

Because of retries, the consumer **must** dedupe by event ID:

```sql
INSERT INTO processed_events (event_id) VALUES ('evt_8f3c2e7a')
ON CONFLICT (event_id) DO NOTHING;     -- if it inserted 0 rows, we've seen it → skip
```

Same idea as idempotency keys, now on the *receiving* side. Without this, a retried `payment.succeeded` could ship the order twice.

15.5 The thundering herd

A spike of events (e.g. a flash sale → 100k `order.created` at once) can **overwhelm consumer endpoints** all at once, or — when many consumers all retry on a synchronized schedule after an outage — hammer the recovering service simultaneously. Mitigations: **jitter** the retry/backoff schedule (randomize so retries spread out), rate-limit outbound deliveries per consumer, and queue + smooth the firehose. (Same root cause as cache-stampede thundering herd — synchronized clients.)

15.6 Webhooks vs polling vs other async patterns

	Polling	Webhooks	Message queue (Kafka/SQS)	SSE/WebSocket
Direction	Client pulls	Server pushes (HTTP)	Pub/sub broker	Server→client over open conn
Latency	Poll interval	Near-real-time	Near-real-time	Real-time

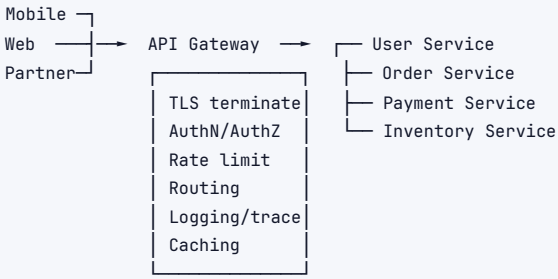
	Polling	Webhooks	Message queue (Kafka/SQS)	SSE/WebSocket
Consumer infra	None (just call)	Must host a public endpoint	Must run a consumer	Persistent connection
Best for	Simple, low-freq	3rd-party integrations	Internal event-driven systems	Live UIs

Interview takeaway: "Webhooks push events to a consumer URL. Three must-haves: **HMAC signature** (with timestamp, constant-time compare) so forged events are rejected; **retries with backoff + jitter** since delivery is at-least-once; and an **idempotent consumer** (dedupe by event ID) so retries don't double-process. Watch for thundering herds on spikes/recovery — jitter the retries."

16 API Gateway & BFF

16.1 API Gateway

A single entry point in front of many backend services that handles **cross-cutting concerns** so each service doesn't reimplement them.



Responsibilities: TLS termination, authentication/JWT validation, rate limiting, request routing, request/response transformation, response aggregation, caching, logging/metrics/-tracing, and API versioning/canary routing. Examples: AWS API Gateway, Kong, Apigee, Envoy, NGINX. **Trade-off:** the gateway centralizes policy (great) but is a potential bottleneck and single point of failure — run it HA, keep per-request logic thin.

16.2 BFF (Backend For Frontend)

One gateway-shaped service **per client type** — a web BFF, a mobile BFF, a TV BFF — each tailoring/aggregating backend calls for *that* client's exact needs.

Why: a generic API forces every client to over-fetch or make many calls; a mobile client on a slow network wants *one* lean, pre-aggregated response. The BFF (popularized by Netflix/-SoundCloud) lets the mobile team own its own aggregation layer and ship without coordinating with the web team. **Trade-off:** more services to maintain and some duplicated logic across BFFs. GraphQL is often used to *implement* a single BFF (clients pick fields instead of needing one BFF per platform) — a common senior talking point: "BFF and GraphQL solve the same over/under-fetching problem differently."

Interview takeaway: Gateway = shared cross-cutting concerns for all clients. BFF = a tailored aggregation layer per client type. GraphQL can replace a fleet of BFFs with one field-selectable endpoint.

17 Designing an API (Worked Example)

> **Brief:** Design the public API for an **Orders / Payments** service (think a small Stripe-like checkout). This exercises every concept above end-to-end. A short URL-shortener example follows.

17.1 Step 1 --- Requirements & consumers

- **Consumers:** partner merchants' *servers* (server-to-server) + their web/mobile front-ends.
- **Functional:** create/read/list/cancel orders; charge a payment for an order; list a customer's orders.
- **Non-functional:** **never double-charge** (idempotency); strong auth; pagination for order history; evolvable; clear errors; rate-limited per merchant.
- **Decision:** REST/JSON over HTTPS (public, partner-facing → universal tooling, cacheable reads). gRPC internally between order-service and payment-service.

17.2 Step 2 --- Resource model

Python

```python
/v1/orders                    collection of orders
/v1/orders/{id}               a single order
/v1/orders/{id}/payment       the payment for an order (action-as-resource)
/v1/customers/{id}/orders     a customer's order history
```

17.3 Step 3 --- Endpoints & schemas

Http

```http
# Create an order — idempotent via key
POST /v1/orders
Authorization: Bearer <merchant-jwt>
Idempotency-Key: 9f1c2e7a-...
Content-Type: application/json

{ "customer_id": "cus_88", "currency": "usd",
  "items": [ { "sku": "BOOK-1", "qty": 2, "unit_amount": 1500 } ] }

→ 201 Created
  Location: /v1/orders/ord_42
  ETag: "ord_42-v1"
{ "id": "ord_42", "status": "pending_payment", "amount": 3000,
  "currency": "usd", "created_at": "2026-06-15T10:00:00Z",
  "_links": { "pay": { "href": "/v1/orders/ord_42/payment", "method": "POST" } } }
```

Http

```http
# Charge the order — also idempotent (this MOVES MONEY)
POST /v1/orders/ord_42/payment
Idempotency-Key: 4b2a...
```

```
{ "source": "card_xyz" }

→ 202 Accepted              # charge is async at the PSP
  Location: /v1/orders/ord_42/payment
{ "status": "processing" }

# Insufficient funds:
→ 422 Unprocessable Entity
{ "error": { "code": "insufficient_funds", "request_id": "req_8f3c",
             "message": "Card has insufficient funds." } }
```

```http
# Read with conditional GET (cache-friendly)
GET /v1/orders/ord_42
If-None-Match: "ord_42-v1"
→ 304 Not Modified          # unchanged → no body

# Cancel — state transition, optimistic concurrency
PATCH /v1/orders/ord_42
If-Match: "ord_42-v1"
{ "status": "cancelled" }
→ 200 OK    (or 409 Conflict if someone changed it first)

# List customer orders — cursor pagination, filtering, sorting
GET /v1/customers/cus_88/orders?status=paid&sort=-created_at&limit=20
→ 200 OK
{ "data": [ ... ],
  "page": { "next_cursor": "eyJjcmVhdGVk...", "has_more": true } }
```

17.4 Step 4 --- Idempotency (the crux)

`POST /orders` and `POST .../payment` carry an `Idempotency-Key`. The server's dedup table (`(merchant_id, idempotency_key) → cached response`) guarantees a retried create returns the *same* `ord_42` and a retried charge charges *once*. (Mechanism + SQL in the Idempotency section.) This is the single most important property of a payments API.

17.5 Step 5 --- Auth

- Merchant **servers** authenticate with OAuth2 **client-credentials** → short-lived **JWT** with scopes (`orders.write`, `payments.write`).
- Merchant **front-ends** use Authorization-Code + PKCE.
- Gateway validates the JWT signature (issuer's public key), checks scope per endpoint → **403** if the token lacks `payments.write`. All over TLS.

17.6 Step 6 --- Errors

Consistent envelope with stable `code` + `request_id` (`insufficient_funds`, `card_declined`, `order_already_paid`). HTTP status for category (`402/422` payment issues, `409` already-paid conflict, `429` throttled). Validation returns all bad fields at once.

17.7 Step 7 --- Rate limiting

Per-merchant token bucket in Redis (e.g. 100 writes/sec burst, steady 50/sec). `429` + `Retry-After` + `X-RateLimit-*` on exceed. Payment endpoints get tighter limits than reads.

17.8 Step 8 --- Versioning

`/v1` in the path; everything additive within v1 (new optional fields, clients ignore unknowns). A breaking change (e.g. `amount` float → `amount_cents` int) ships as a *new field* in v1, then drops the old one only in `/v2`, with `Deprecation`/`Sunset` headers and a 6-month window.

17.9 Step 9 --- Async + webhooks

The charge is async (`202` + poll, or better, a **webhook**): when the PSP settles, we POST `payment.succeeded` to the merchant's URL, HMAC-signed, with retries + at-least-once delivery → merchant consumer dedupes by event ID.

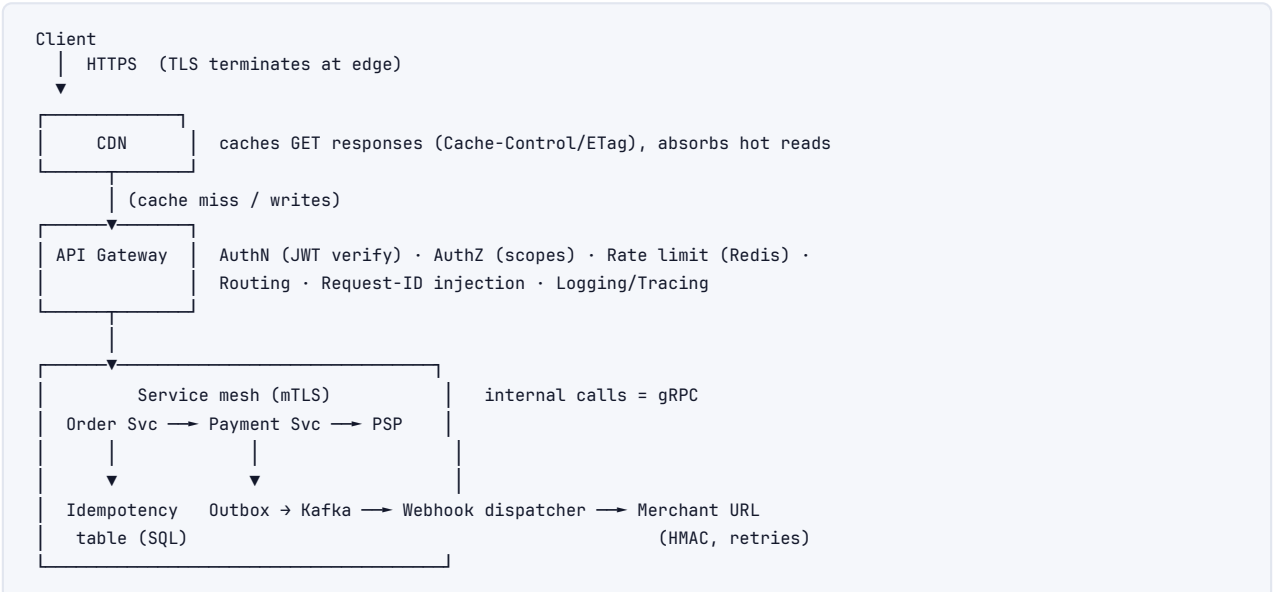
17.10 Mini-example --- URL Shortener API

```http
POST /v1/links
Idempotency-Key: ...
{ "long_url": "https://example.com/very/long", "custom_alias": "promo" }
→ 201 Created
  Location: /v1/links/abc123
{ "code": "abc123", "short_url": "https://acme.co/abc123",
  "long_url": "https://example.com/very/long", "created_at": "..." }

GET /abc123          → 301 Moved Permanently   Location: https://example.com/very/long
                       (301 is cacheable → CDN absorbs hot links; use 302 if you need
                        per-click analytics so the browser re-hits you each time)

GET /v1/links?limit=20&cursor=...     → cursor-paginated list of the user's links
DELETE /v1/links/abc123               → 204 No Content (idempotent)
```

Note the deliberate status choice: **301 vs 302** for the redirect is a real trade-off — 301 lets browsers/CDNs cache the redirect (fast, but you lose per-click tracking); 302 forces a round trip every click (slower, but enables analytics). Calling that out wins points.

18 Architecture / Diagrams

18.1 Request lifecycle through the stack



18.2 Idempotent POST flow

```
Client ──POST /payments, Idempotency-Key:K──►  Gateway ──► Payment Svc
                                                              │
                              INSERT (merchant,K) ON CONFLICT ──┤
                                ├─ inserted (first time) ──► charge card ──► store resp ──► 201
                                ├─ conflict, completed ─────► return STORED response (no charge)
                                └─ conflict, in_progress ──► 409 Conflict (retry later)
```

18.3 Cursor pagination walk

```
page 1:  GET /events?limit=3
         rows = [E9, E8, E7]   next_cursor = enc(E7.created_at, E7.id)
page 2:  GET /events?limit=3&cursor=enc(E7...)
         WHERE (created_at,id) < (E7...)  →  [E6, E5, E4]
         (new row E10 inserted at top between calls → does NOT shift this walk)
```

18.4 OAuth2 Authorization-Code + PKCE (compact)

```
User → App: login
App: verifier=rand; challenge=SHA256(verifier)
App → AuthServer: /authorize?...&code_challenge=challenge
AuthServer → User: consent → redirect ?code=CODE
App → AuthServer: /token  code=CODE & code_verifier=verifier
AuthServer: assert SHA256(verifier)==challenge → {access_token, refresh_token}
App → API: Bearer access_token
```

19 Real-World Examples

Stripe — the canonical well-designed API. Idempotency keys on every POST (`Idempotency-Key` header), **cursor** pagination (`starting_after`/`ending_before`), **date-based** versioning (`Stripe-Version: 2024-04-10` pins a behavior snapshot), rich error objects with stable `type`/`code`/`decline_code`, and HMAC-signed webhooks. If you cite one API in an interview, cite Stripe.

GitHub — REST (v3) *and* GraphQL (v4) side by side; they added GraphQL to fix over/under-fetching of the deeply nested issue/PR/repo graph. Cursor pagination via `Link` headers (`rel="next"`), conditional requests with ETags (304s don't count against rate limits), and `X-RateLimit-*` headers.

AWS — Amazon's 2002 internal mandate ("all functionality exposed *only* via service interfaces") is why AWS exists as a product. APIs are signed (SigV4 — request signing, not just bearer tokens), strongly versioned, and designed for partners from day one.

Google APIs (Maps, Cloud) — follow the public AIP/API Design Guide: resource-oriented, standard methods (`List`/`Get`/`Create`/`Update`/`Delete`), `pageToken`/`pageSize` pagination, field masks for partial reads/updates, gRPC + REST transcoding from one `.proto`.

Slack/Meta — Meta runs GraphQL at massive scale with persisted queries and DataLoader-style batching; Slack's webhooks (Events API) require URL verification + signature checks, a textbook webhook security setup.

20 Real-Life Analogies

One restaurant — every API concept is part of how the kitchen serves diners.

Concept	Analogy
API contract	The **menu**: it tells you exactly what you can order and what you'll get, without you ever entering the kitchen
REST resources/verbs	Menu items are **nouns** (a ”steak”); you act on them with standard verbs — order, modify, cancel — not with a different word per dish
Idempotency key	Writing your **order number** on the ticket: if the waiter loses it and you re-hand it, the kitchen sees the same number and makes your meal **once**, not twice
GET (safe)	**Reading the menu** — you can do it a hundred times and nothing in the kitchen changes
PUT (idempotent)	Telling the chef ”make the table's order *exactly* this” — repeating it leaves the same final order
POST (not idempotent)	Placing a **new order** — say it twice and two meals come out
Pagination (cursor)	”Bring the next 5 dishes **after** the one I just got” — even if new dishes are added to the kitchen, your sequence never repeats or skips
Pagination (offset)	”Bring dishes 41–45” — if a dish is inserted at the front, the numbering shifts and you get one twice
Versioning	Printing a **new menu** for breaking changes, while keeping today's menu valid for guests already mid-meal
Auth (OAuth/scopes)	A **valet ticket**: it lets the valet fetch *your* car only, not drive off in anyone else's — least privilege
JWT	A **wristband** at a festival — staff read it to know your access tier without phoning the box office; but you can't un-issue a wristband already on a wrist (revocation is hard)
Rate limiting	The **”two drinks per round” rule** at the bar — protects the bartender from one guest monopolizing service
429 + Retry-After	”We're slammed — come back in 5 minutes” instead of just ignoring you
Error envelope	A clear **”dish unavailable: out of salmon”** note, with a ticket number, instead of a blank stare
gRPC	The **kitchen's internal shorthand** — terse, fast, only the line cooks understand it; never handed to a diner
GraphQL	A **build-your-own plate** counter — you point at exactly the items you want and get one plate, no more, no less
Webhook	The **buzzer they hand you** — instead of you asking ”ready yet?” every minute, it buzzes when your order is done
API Gateway	The **maître d'** at the door — checks reservations (auth), seats you (routing), enforces the dress code (rate limits) before you reach any table
BFF	Separate **kids' / vegan / tasting menus** — each guest type gets a tailored menu instead of one giant generic one

21 Memory Tricks / Mnemonics

Safe vs Idempotent (the table you must recall):

> "**G**ET is **g**entle (safe + idempotent), **P**UT/**D**ELETE are **p**owerful but **r**epeatable (idempotent, not safe), **P**OST/**P**ATCH are **p**erilous (neither)."

Status code families: "**1** Info, **2** Yes, **3** Go elsewhere, **4** Your fault, **5** My fault."
401 vs 403: "**401** = *who* are you? (un-authenticated). **403** = I know you, **for**bidden anyway."
400 vs 422: "**400** = can't **read** it. **422** = read it, **don't like** it."
Idempotency keys: "**K**ey makes retries **K**osher — at-least-once delivery → exactly-once effect."
Pagination choice: "**O**ffset is **O**ld and **O**(n); **C**ursor is **C**onstant and **C**orrect under writes."
Versioning rule: "**A**dd, don't **A**lter" — additive = safe, altering = breaking.
REST maturity (Richardson): "**P**OX → **R**esources → **V**erbs → **H**ypermedia" = "**P**lease **R**est **V**ery **H**ard." (Most APIs stop at V = Level 2.)
gRPC RPC types: "**U**nary, **S**erver-stream, **C**lient-stream, **B**idi" = "**U**ncle **S**am **C**ooks **B**reakfast."
Rate-limit algorithms: "**T**oken bursts, **L**eaky smooths, **F**ixed edges-double, **S**liding fixes it."
Webhook must-haves: "**S**ign it (HMAC), **R**etry it (backoff), **D**edupe it (idempotent consumer)" = **SRD**.
OAuth in one line: "**Authorize** → **Code** → **Token** → **Access**" (and PKCE = *Prove* you started the flow).

22 Common Interview Questions

22.1 Q1: What's the difference between idempotent and safe HTTP methods, and why does it matter?

Model answer: *Safe* = no observable side effects (read-only): GET, HEAD, OPTIONS. *Idempotent* = N identical calls leave the same server state as one: GET, PUT, DELETE (and HEAD/OPTIONS). POST and PATCH are neither in general. It matters because networks force **retries** — a client times out and resends. Idempotent operations can be retried safely; POST cannot, which is the double-charge bug. PUT is idempotent because it *sets* full state; DELETE because the end state (gone) is unchanged; POST isn't because it *creates a new* resource each time. PATCH depends — a "set field" patch is idempotent, a "+10" delta isn't.

Follow-ups: *How do you make POST idempotent?* → idempotency keys + dedup table. *Is DELETE returning 404 on the second call a violation?* → No; idempotency is about *state*, not status code.

22.2 Q2: Walk me through how you'd add an idempotency key to a payment endpoint.

Model answer: Client generates a UUID per logical payment and sends `Idempotency-Key` with every retry. Server keeps a table keyed `(api_key, idempotency_key)` storing a request-body hash, state, and cached response. On each request: atomically `INSERT ... ON CONFLICT`. If it inserts, this is the first time → execute the charge, store the response, return it. If it conflicts and state=`completed`, return the *stored* response verbatim (no second charge). If conflict and the stored body hash differs, reject 422 (key reused for a different request). If conflict and state=`in_progress`, return 409. Keys get a TTL (~24h). This turns at-least-once delivery into exactly-once *effects*.

Follow-ups: *Why hash the body?* → catch a client reusing a key for a different request. *Where's the race?* → the atomic insert is the gate; two concurrent retries can't both win it. *DB vs Redis for the table?* → DB for durability of the result; the unique constraint must be transactional.

22.3 Q3: Offset vs cursor pagination --- which and why?

Model answer: Offset (`LIMIT 20 OFFSET 40`) is simple and allows random access ("page 7"), but at large offsets the DB scans-and-discards O(offset) rows (page 50k times out), and it's *unstable* under writes — a row inserted at the top shifts every page, causing duplicates/skips. Cursor/keyset pagination remembers the last item's sort key and queries `WHERE (created_at, id) < (...)`, which **seeks via index** in O(page size) regardless of depth and is stable under inserts. The cursor is base64-opaque so I can change its internals. The cost: no random access and total counts are expensive. For a feed at scale, cursor wins.

Follow-ups: *Why include `id` in the cursor?* → tiebreaker for a total order when `created_at` collides. *How do you paginate backwards?* → reverse the comparison and ordering. *Total count?* → expensive; often approximate or omit.

22.4 Q4: How do you version an API and handle breaking changes?

Model answer: Version only on *breaking* changes; ship everything else additively, relying on clients ignoring unknown fields (Postel's Law). URL path versioning (`/v1`) by default — most debuggable. Non-breaking: new optional fields, new endpoints, new optional params. Breaking: removing/renaming fields, changing types/semantics, making optional fields required. For a breaking change I add a *new* field in v1 alongside the old, deprecate the old, and only drop it in v2 — with `Deprecation`/`Sunset` headers, a 6-month window, client monitoring by API key, and finally `410 Gone`.

Follow-ups: *URL vs header vs media-type versioning trade-offs?* → table. *Stripe's date versioning?* → pins a behavior snapshot per client. *How do you know who's still on v1?* → log by API key.

22.5 Q5: 401 vs 403 vs 422 vs 400 --- when do you return each?

Model answer: **400** = unparseable/malformed request (truncated JSON, missing required param). **422** = syntactically valid but semantically invalid (negative age, bad email format) — I parsed it, the *content* breaks a rule. **401** = not authenticated (no/invalid credentials) — *who are you?* **403** = authenticated but not authorized (valid token, wrong scope/permission). The HTTP status gives the category; a stable `code` in the body gives specifics clients branch on.

Follow-ups: *Read-only token tries DELETE?* → 403. *Should 404 ever be used instead of 403?* → yes, to avoid leaking that a resource exists to an unauthorized caller. *Where does 409 fit?* → conflict with current state (duplicate key, stale ETag write).

22.6 Q6: REST vs gRPC vs GraphQL --- when each?

Model answer: REST/JSON for **public/partner** APIs — universal tooling, HTTP caching, human-debuggable. gRPC (Protobuf + HTTP/2) for **internal service-to-service** — binary, typed, streaming, fast, but no native browser support (needs grpc-web + Envoy). GraphQL when you have **many diverse clients** fetching nested data and want to kill over/under-fetching — at the cost of harder caching and the N+1 problem. Lead with *who the consumer is*.

Follow-ups: *Why can't browsers do gRPC?* → no access to HTTP/2 framing/trailers from JS. *GraphQL caching?* → one POST endpoint defeats HTTP caching; use client-side normalized cache + persisted queries. *Protobuf compatibility?* → field numbers, not names, are the wire identity.

22.7 Q7: What's the N+1 problem in GraphQL and how do you fix it?

Model answer: Resolving a list field then a nested field per item fires 1 + N queries — e.g. fetch 5 orders (1 query), then items per order (5 queries). **DataLoader** fixes it by batching: it collects all the keys requested within one event-loop tick and issues a single `WHERE order_id = ANY(...)` query, then maps results back per key, and dedupes repeated keys. So 1 + 1 instead of 1 + N.

Follow-ups: *Does this apply to REST?* → yes, the same batching idea (the under-fetching problem GraphQL was built to solve). *Protecting against expensive queries?* → depth limiting, complexity scoring, persisted queries, pagination caps.

22.8 Q8: How do you rate-limit an API? Compare the algorithms.

Model answer: Token bucket (refill rate r, capacity B) allows bursts up to B then steadies to r — the common default. Leaky bucket drains at a fixed rate, perfectly smooth but no bursts — good to protect a fragile downstream. Fixed-window counter is simplest (`INCR+EXPIRE`) but has the boundary-burst flaw (2× at the window edge). Sliding-window log is exact but O(n) memory. Sliding-window counter approximates the window in O(1) and fixes the boundary burst — best general default. Counters live in **Redis** so limits are global across gateway nodes. On exceed: 429 + `Retry-After` + `X-RateLimit-*`.

Follow-ups: *Per what dimension?* → API key, user, IP, endpoint; tiered by plan. *Why Redis not in-memory?* → otherwise a client gets N×instances. *Distributed correctness?* → atomic Lua scripts for the token-bucket update.

22.9 Q9: How do you design a secure webhook system?

Model answer: Sender POSTs the event to the consumer's registered URL. Three must-haves: (1) **HMAC signature** over the raw body + a timestamp, sent in a header; the consumer recomputes with the shared secret using a constant-time compare and rejects forged/old payloads — defeats forgery and replay. (2) **Retries with exponential backoff + jitter** since delivery is at-least-once and consumers go down. (3) **Idempotent consumer** — dedupe by event ID (`INSERT ... ON CONFLICT DO NOTHING`) so retries don't double-process. Consumers should 2xx fast and process async. Watch for thundering herds on spikes/recovery — jitter the retries.

Follow-ups: *Why sign over the raw body?* → re-serializing JSON changes bytes. *Why include the timestamp in the signature?* → otherwise a captured valid payload replays forever. *At-least-once vs exactly-once?* → can't guarantee exactly-once delivery; you achieve exactly-once *effects* via consumer idempotency.

22.10 Q10: What is a JWT, and what are its security pitfalls?

Model answer: A JWT is a signed, self-contained token: `header.payload.signature`, base64url-joined. Claims (`sub`, `exp`, `aud`, `scope`) sit in the payload; the resource server verifies the signature with the issuer's public key (RS256) — *no DB lookup*, ideal for stateless scaling. Pitfalls: the payload is **not encrypted** (don't put secrets in it); **revocation is hard** (valid until `exp` — mitigate with short expiry + refresh tokens or a `jti` denylist); the **`alg:none`** and **RS256→HS256 key-confusion** attacks (always pin the expected algorithm server-side); clock skew; and header bloat since it rides every request.

Follow-ups: *Stateless JWT vs server session?* → JWT = fast stateless validation, weak revocation; session = instant revocation, needs a lookup. *How to log a user out?* → short expiry + revoke the refresh token; denylist the `jti`.

22.11 Q11: Design the API for a URL shortener.

Model answer: `POST /v1/links {long_url, custom_alias?}` → 201 with `code` + `short_url`, idempotency-keyed so a retry doesn't mint two codes. `GET /{code}` → 301 (cacheable, CDN-friendly) or 302 (forces a round trip → enables per-click analytics). `GET /v1/links?limit&cursor` cursor-paginated. `DELETE /v1/links/{code}` → 204, idempotent. Auth via bearer token; rate-

limit creation per user. The interesting trade-offs: 301 vs 302 (caching vs analytics), code generation (counter+base62 vs hash vs pre-generated pool), and idempotency on create.

Follow-ups: *301 vs 302 trade-off?* → caching vs tracking. *How to avoid code collisions?* → pre-generated pool or base62(counter). *Custom alias conflict?* → 409.

22.12 Q12: How do you report errors consistently across an API?

Model answer: A single envelope on every error: a stable machine-readable `code` (clients branch on it), a human `message`, a `request_id` that also appears in logs, optional `details[]` for field-level validation. HTTP status gives the category, `code` the specifics. Return *all* validation errors at once, use per-item results (or 207) for batch partial failures, and **never leak stack traces/SQL** in 500s. Optionally adopt RFC 7807 (`application/problem+json`).

Follow-ups: *Why a `code` if you have the status?* → status is too coarse (many 422s with different causes). *RFC 7807?* → standardized `type/title/status/detail/instance`. *Partial failure?* → per-item status array.

23 Senior-Level Discussion Points

1. Idempotency keys vs natural idempotency. The cleanest design avoids keys entirely by modeling operations as idempotent: prefer `PUT` with a client-supplied ID over `POST` with a server-assigned one, prefer absolute state (`set balance=100`) over deltas (`add 10`). Idempotency keys are the fallback when the operation is inherently a non-idempotent create. Senior signal: *reach for natural idempotency first, keys second.*

2. Exactly-once is a lie at the delivery layer. You cannot guarantee exactly-once *delivery* over an unreliable network (it's provably impossible without coordination). What you achieve is at-least-once delivery + idempotent processing = exactly-once *effects*. Saying "exactly-once delivery" unqualified is a red flag; "exactly-once effects via idempotency" is the correct framing.

3. Pagination + total counts at scale. Cursor pagination gives O(1) pages but `COUNT(*)` over a huge filtered set is O(n) and kills the DB. At scale you either omit totals, return approximate counts (e.g. from stats/`reltuples`), or maintain a counter. Interviewers probe this when you claim cursor pagination "scales."

4. Versioning is an org problem, not a syntax problem. The hard part isn't `/v1` vs a header — it's the *transformation layer* that lets one backend serve multiple contract versions, plus the discipline to deprecate. Stripe's per-date versioning means maintaining a chain of request/response transformers. Mention the maintenance cost.

5. JWT revocation and the stateless paradox. JWTs are loved for being stateless, but real systems need logout/ban/key-rotation → revocation → a lookup → no longer stateless. The honest answer is a hybrid: short-lived access JWTs (stateless validation) + a stateful refresh-token / denylist for revocation. Acknowledging this tension is senior-level.

6. Rate limiting in a distributed gateway. A naive per-instance counter lets a client get `limit × N` across N gateway nodes. Correct designs centralize the counter (Redis with atomic Lua) or use a quota-distribution scheme (each node gets a slice of the global budget). Also: rate-limit *cost*, not just request count — one expensive query may equal 100 cheap ones.

7. Backward/forward compatibility as a contract. Protobuf field numbers and "ignore unknown JSON fields" are the *same idea*: decouple wire identity from names and tolerate unknowns so producers and consumers can deploy independently. This is what makes zero-downtime rolling deploys of API changes possible.

8. The gateway as a coupling/SPOF risk. Centralizing auth, rate limiting, and routing in a gateway is powerful but creates a single dependency every request flows through. Keep gateway logic thin and stateless, run it HA, and resist putting business logic there (it becomes a distributed-monolith chokepoint).

9. HATEOAS in reality. Full hypermedia is rarely worth it (verbose, most clients hard-code paths). But the *idea* survives in `next`/`prev` pagination links and in returning available state-transition actions. Know when the cost is justified (long-lived, loosely-coupled clients) vs not.

10. GraphQL's security surface. Giving clients query power means giving them a DoS vector. Depth limiting, complexity budgets, persisted queries (allowlisting), and disabling introspection in production are mandatory — "GraphQL is flexible" must be paired with "and here's how I stop it melting the DB."

24 Typical Mistakes Candidates Make

1. **Putting verbs in URLs.** `POST /createOrder`, `GET /getUser`. The verb is the HTTP method; the URL is a noun. `POST /orders`, `GET /users/42`.
2. **Confusing 401 and 403.** 401 = unauthenticated (send credentials); 403 = authenticated but forbidden. The single most common status-code mistake.
3. **Forgetting idempotency on POST/payments.** Claiming "the client just retries" without addressing the double-charge. Always raise idempotency keys for unsafe creates.
4. **Defaulting to offset pagination at scale.** Not knowing why `OFFSET 1000000` is slow (scan-and-discard) or that it's unstable under writes. Reach for cursor/keyset and write the `WHERE (created_at, id) < (...)` query.
5. **Saying "exactly-once delivery."** Impossible over an unreliable network. The correct phrase is "at-least-once delivery + idempotent consumer = exactly-once effects."
6. **Treating PATCH as idempotent.** A delta PATCH (`+10`) applied twice is wrong. Only "set field" patches are idempotent.
7. **Putting secrets in a JWT.** The payload is base64, not encrypted — anyone can decode it. Also forgetting that JWTs can't be easily revoked.
8. **Versioning everything.** Bumping to `/v2` for an additive change. Additive changes need no version if clients ignore unknown fields.
9. **Inconsistent / leaky errors.** Different error shapes per endpoint, no stable `code`, no `request_id` — or worse, returning stack traces in 500s.
10. **No rate-limit headers.** Returning 429 with no `Retry-After`/`X-RateLimit-*`, so clients can't back off intelligently.
11. **Webhook without signature verification or idempotency.** Trusting a public POST endpoint, or double-processing retried events. Always HMAC-verify and dedupe by event ID.
12. **Recommending gRPC for a browser/public API.** Forgetting browsers can't speak native gRPC (need grpc-web + a proxy). gRPC is for internal services.
13. **Deep nesting in URLs.** `/users/1/orders/2/items/3/reviews/4` — hard to authorize and build. Nest one level, then address sub-resources directly.
14. **N+1 in GraphQL/resolvers** with no DataLoader, and exposing GraphQL with no depth/complexity limits.

25 How This Connects to Other Topics

25.1 Computer Networks (03)

› HTTP methods, status codes, headers, content negotiation, caching (ETag/Cache-Control), CORS, HTTP/2 vs /3, and TLS are the *transport* this API layer sits on. gRPC's advantages are HTTP/2's advantages.

25.2 System Design (05)

› ”Define the API” is Step 3 of every design interview. Rate limiting, API gateway, idempotency, and caching all appear there. The endpoints you sketch surface the data model.

25.3 Distributed Systems (07)

› Idempotency, at-least-once delivery, exactly-once effects, the outbox pattern, and webhook retries are distributed-systems reliability patterns expressed at the API boundary.

25.4 Databases (08)

› Pagination is a *query* problem (offset scan vs index seek). Idempotency tables, optimistic concurrency via ETags (≈ row versions), and unique constraints for dedup are DB mechanisms.

25.5 Caching (15)

› `Cache-Control`/`ETag`/`304` are the HTTP cache protocol. CDNs cache GET responses; 301 redirects are cacheable. GraphQL's caching difficulty is a direct consequence of using one POST endpoint.

25.6 Message Queues (16)

› Webhooks, async `202`-then-poll, and event-driven APIs sit on queues (Kafka/SQS). The outbox pattern bridges a DB write and an event publish atomically.

25.7 Security (10)

› OAuth2, JWT, mTLS, scopes, HMAC signing, and ”never leak internals in errors” are the API security surface. AuthN/AuthZ on every endpoint; TLS everywhere.

25.8 Observability (18)

› `request_id` correlation, rate-limit metrics, error-code dashboards, and tracing through the gateway are how you operate an API in production.

26 FAANG Interview Tips

1. Always start from the consumer. ”Who calls this — a partner server, a browser, an internal service?” That single question chooses REST vs gRPC vs GraphQL and frames everything else.

2. Volunteer idempotency unprompted. For any create/charge, say ”I'll make this idempotent with a key so retries don't double-process” and sketch the dedup table. It's the strongest single signal in an API round.

3. Write a real query for pagination. Don't just say ”cursor pagination” — write `WHERE (created_at, id) < (...) ORDER BY created_at DESC, id DESC LIMIT n` and explain the O(1) index seek and write-stability. This separates you instantly.

4. Name status codes precisely. 201+Location for creates, 202 for async, 204 for deletes, 401 vs 403, 409 for conflicts, 422 for validation, 429 for limits, 410 for sunset versions. Precision reads as experience.

5. Treat evolution as a process. ”Additive by default, version only on breaking changes, `Deprecation`/`Sunset` headers, 6-month window, then 410.” Shows you've operated an API, not just designed one.

6. Pair every choice with its cost. ”GraphQL kills over-fetching *but* caching is harder and you must limit query cost.” ”JWTs are stateless *but* hard to revoke.” Trade-offs are the currency of senior interviews.

7. Security is reflexive. "AuthN before AuthZ, scopes for least privilege, TLS everywhere, HMAC-signed webhooks, never leak internals in 500s." Say it without being asked.

8. Know the canonical APIs. Cite Stripe (idempotency keys, cursor pagination, date versioning, rich errors) and GitHub (REST+GraphQL, ETag/304, `Link` pagination). Concrete references beat abstractions.

9. For Amazon: invoke the API mandate and service ownership; emphasize idempotency, backward compatibility, and designing for external partners.

10. For Meta/Google: be fluent in GraphQL (N+1/DataLoader, persisted queries) and resource-oriented design / Protobuf evolution respectively — these are *their* technologies and interviewers know them deeply.

27 Revision Cheat Sheet

27.1 10-Minute Summary

Methods (safe/idempotent): GET (safe+idem), HEAD/OPTIONS (safe+idem), PUT (idem), DELETE (idem), POST (neither), PATCH (neither). Idempotency = safe retries.

Status codes: 200 OK · 201 Created (+Location) · 202 Accepted (async) · 204 No Content · 301 moved · 304 Not Modified (ETag) · 400 malformed · 401 unauthenticated · 403 forbidden · 404 not found · 409 conflict · 410 gone · 422 semantic-invalid · 429 rate-limited (+Retry-After) · 500/503.

Idempotency: `Idempotency-Key` → dedup table `(api_key,key)` → atomic INSERT ON CONFLICT → cached response on retry. At-least-once delivery + idempotency = exactly-once effects.

Pagination: offset = O(offset), unstable under writes, random access. Cursor/keyset = `WHERE (created_at,id) < (...)`, O(page size), stable, opaque base64 cursor, no random access. Cursor for feeds at scale.

Versioning: additive by default (clients ignore unknown fields); version only on breaking changes; URL path default; `Deprecation`/`Sunset` headers → 410 Gone.

Auth: API keys (server-to-server), OAuth2 (auth-code+PKCE for users, client-credentials for services), JWT (header.payload.signature, stateless, hard to revoke — short exp + refresh), mTLS (service mesh), scopes (least privilege → 403 on missing).

Rate limiting: token bucket (bursts), leaky bucket (smooth), fixed window (edge-double), sliding-window-counter (best default). Redis-shared counters. 429 + Retry-After + X-RateLimit-*.

Errors: consistent envelope, stable `code`, `request_id`, all validation errors at once, never leak internals, RFC 7807 (`problem+json`).

gRPC: Protobuf + HTTP/2, 4 RPC types (unary/server-stream/client-stream/bidi), binary+typed+streaming, internal services, no native browser (grpc-web+Envoy). Field *numbers* = wire identity.

GraphQL: client picks fields (no over/under-fetch), resolvers, **N+1 → DataLoader batching**, limit depth/complexity + persisted queries, caching harder (one POST endpoint).

Webhooks: push events, HMAC-sign (timestamp + constant-time compare), retry w/ backoff+jitter (at-least-once), idempotent consumer (dedupe by event ID).

27.2 Quick-Reference Tables

Method	Safe	Idempotent	Typical status
GET			200 / 304
POST			201 / 202

Method	Safe	Idempotent	Typical status
PUT			200 / 204
PATCH			200
DELETE			204 / 404

Style	Transport	Format	Best for
REST	HTTP/1.1-2	JSON	Public/partner CRUD
gRPC	HTTP/2	Protobuf	Internal microservices
GraphQL	HTTP	JSON	Diverse clients, nested data

Pagination	Deep-page cost	Stable	Random access
Offset	O(offset)	No	Yes
Cursor	O(page)	Yes	No

27.3 Key Numbers / Defaults to Cite

Item	Value
Idempotency key TTL	~24h
JWT access token expiry	5–15 min (+ refresh token)
Deprecation window	6–12 months → 410 Gone
Rate-limit headers	`Retry-After`, `X-RateLimit-{Limit,Remaining,Reset}`
Conditional GET	`ETag` + `If-None-Match` → 304
Async pattern	202 Accepted + status URL to poll

27.4 FAANG Top-10 (most likely to appear)

1. Safe vs idempotent methods + why retries need them
2. Idempotency keys + dedup table mechanism (payments)
3. Offset vs cursor pagination (+ the SQL)
4. Versioning & breaking-change handling
5. 401/403/422/409 status-code precision
6. REST vs gRPC vs GraphQL trade-offs
7. GraphQL N+1 + DataLoader
8. Rate-limiting algorithms + 429/Retry-After
9. OAuth2 + JWT + scopes (and JWT revocation)
10. Webhook security (HMAC + retries + idempotent consumer)